Producto Integrador de Aprendizaje (PIA)

Reporte final · Sistema de lógica de negocio NovaRent

Nombres:	Matrícula:
Arturo Sebastian Pérez Zapata	2142942
Cesar Maximiliano Peña Mendoza	2144135
Diego Alberto Sierra Mendoza	2143733
Nombre del curso: Componentes Lógica de negocio	Nombre del profesor: Miguel Ángel Gómez Marroquín
Actividad: Producto Integrador de Aprendizaje (PIA) · Reporte final	Fecha: 06/11/2026
Repositorio: https://github.com/<usuario>/<repositorio>	Aplicación: https://<servicio>.onrender.com/docs

1. Resumen ejecutivo

NovaRent renta automóviles por día en varias sucursales. Este proyecto implementa su **núcleo de lógica de negocio**: reservación de una categoría (sin fijar unidad), entrega con checklist fotográfico y firma, contrato, extensiones con autorización por rol, devolución, cargos automáticos por kilometraje, combustible, retraso y daños, disputas resueltas con evidencia, liberación del depósito de garantía, limpieza y reportes. Se construyó con la arquitectura diseñada en la Unidad 3 (*Domain Model + Service Layer*, Repository, Unit of Work, Domain Events, Specification y máquinas de estado) y se expone mediante una API REST documentada con OpenAPI.

Indicador	Resultado
Historias de usuario con criterios Gherkin	20 (mínimo exigido: 15)
Código	≈ 4,500 líneas en *src/* y ≈ 2,000 en *tests/*
Pruebas automatizadas	354 pasan en SQLite y 356 en PostgreSQL 16 real
Cobertura (líneas y ramas)	98.5 % (mínimo exigido: 80 %)
Calidad estática	*ruff check* sin errores
API	32 rutas OpenAPI con contrato de errores 200/201/400/401/403/404/409/422
Decisiones de arquitectura	10 ADR
Paquete de datos	120 casos y 20 transiciones ejecutados, 0 diferencias contra un oráculo independiente
Entrega	Docker Compose (DB + App + Tests), GitHub Actions y Blueprint de Render

2. Contexto y requerimientos (BRD)

Problema. La disponibilidad real de vehículos no está actualizada, los daños iniciales se documentan de forma inconsistente (lo que genera reclamaciones por cargos) y la coordinación entre devolución, limpieza y siguiente renta es manual y lenta. **Oportunidad.** Un proceso digital con checklist y evidencia, cálculo automático de cargos y visibilidad de flota reduce reclamaciones y mejora la rotación.

Stakeholders y alcance

Stakeholder	Responsabilidad	Rol en el sistema
Cliente	Renta, paga cargos y puede disputarlos	CLIENTE
Agente de renta	Valida datos, entrega y recibe la unidad	AGENTE
Personal de patio	Inspecciona y captura el checklist	PATIO

Stakeholder	Responsabilidad	Rol en el sistema
Limpieza	Prepara la unidad para la siguiente renta	LIMPIEZA
Administración	Autoriza excepciones, cargos y cortesías; reportes	ADMINISTRACION
Sistema	Jobs de no-show y liberación de depósitos	SISTEMA

Dentro del alcance: reservación, entrega y devolución con checklist, cálculo de cargos, excepciones (falta de unidad, upgrades, extensiones, disputas), depósito de garantía y reportes. **Fuera:** facturación, lealtad (salvo prioridad), mantenimiento y telemetría GPS.

Necesidades del negocio

Código	Necesidad	Cómo se resuelve en el sistema
N1	Disponibilidad en tiempo real	Consulta viva: flota − reservaciones − contratos traslapados
N2	Documentación confiable de daños	No existe checklist sin foto con sello de tiempo; firmado = inmutable
N3	Cálculo automático de cargos	*PoliticaCargos* pura, parametrizable y probada en sus bordes
N4	Excepción «categoría sin unidad»	Upgrade sin costo a la inmediata superior, lista de espera o reembolso
N5	Autorización por rol	Matriz de permisos + invariantes de rol dentro de los aggregates
N6	Depósito de garantía	Retención al abrir; liberación a 5 días hábiles sin incidencias
N7	Coordinación devolución-limpieza	Evento *VehiculoDevuelto* abre la tarea; completarla libera la unidad
N8	Menos reclamaciones	Disputas resueltas comparando evidencia de entrega y devolución
N9	Reportes	Ocupación, ingresos y rotación con SQL directo
N10	Trazabilidad	Máquinas de estado, eventos y cierre formal del contrato

3. Análisis: historias de usuario y reglas de negocio

Épica	Historias	Endpoint principal
Reservación	US01 disponibilidad · US02 reservar · US03 cancelar y penalización · US04 no-show	GET /disponibilidad · POST /reservaciones
Entrega	US05 documentos y pago · US06 checklist con fotos · US07 firma digital	POST /reservaciones/{folio}/entrega · /inspecciones
Excepción	US08 upgrade sin costo · US09 rechazo (lista de espera o reembolso) · US10 cortesía	POST …/upgrade/respuesta · /cortesia
Extensión	US11 hasta 48 h · US12 más de 48 h (Administración)	POST /contratos/{id}/extensiones
Devolución y cargos	US13 checklist de devolución · US14 km · US15 combustible · US16 retraso · US17 disputas	POST /contratos/{id}/devolucion
Cierre y reportes	US18 limpieza y disponibilidad · US19 reportes · US20 liberación de depósito	GET /reportes/ocupacion

Regla	Valor	Implementación
RN-01 Cancelación sin penalización	≥ 24 h antes	*CancelacionSinPenalizacion*
RN-02 No-show	2 h tras el inicio	*NoShowVencido*
RN-03/04 Kilometraje	200 km por día; $3 MXN por km extra	*PoliticaCargos*
RN-05 Combustible	Cargo si falta más de 1/8	*CombustibleFaltante*
RN-06 Devolución tardía	29 min de tolerancia; por hora; tope de 1 día	*DevolucionTardia*
RN-07/08/09 Extensión	Agente hasta 48 h; más, Administración; ≥ 2 h de anticipación	*PoliticaExtension*

Regla	Valor	Implementación
RN-10 Depósito	5 días hábiles, sin incidencias	*deposito_liberable()* + calendario
RN-11/12 Upgrades	Sin costo, solo categoría inmediata superior; cortesía solo Administración	*PoliticaUpgrade*

Supuestos agregados en el PIA (el BRD no fija estos valores; son parámetros configurables que deben validarse con el negocio): penalización por cancelación tardía = 1 día de tarifa; tarifa por octavo de combustible = $50; cargo por hora de retraso = tarifa diaria ÷ 24; monto del depósito por categoría; días facturables en bloques de 24 h.

Procesos BPMN

Los cinco procesos (reservación, entrega y excepciones, extensión, devolución y cargos, cierre) están modelados en *docs/bpmn.md* en formato Mermaid y se incluyen en el Anexo A.

4. Arquitectura y diseño

Se aplica *Domain Model + Service Layer* (ADR-001) en cuatro capas con dependencias hacia el dominio. El dominio no importa FastAPI ni SQLAlchemy; la aplicación depende de **puertos** (*UnidadDeTrabajo, Reloj, PasarelaPago, AlmacenEvidencia, BusEventos, CalendarioHabil*) que la infraestructura implementa. Los diagramas de capas, clases, estados, secuencia, ER y despliegue se entregan en Mermaid (Anexo A).

Capa	Carpeta	Responsabilidad
API	src/api	HTTP ⇄ comandos/DTO; mapa único de errores a códigos
Aplicación	src/application	Casos de uso, transacciones, permisos por rol, publicación de eventos
Dominio	src/domain	Aggregates, Value Objects, Specifications, políticas, máquinas de estado, eventos
Infraestructura	src/infrastructure	Repositorios, Unit of Work, SQLAlchemy, pasarela, evidencia, calendario, bus

Aggregates y patrones

Aggregate	Invariantes principales	Estados
Reservacion	Garantiza categoría, no unidad; cancelación y no-show por horas; upgrade solo a la inmediata superior	CREADA, UPGRADE_OFRECIDO/ACEPTADO, EN_LISTA_ESPERA, CANCELADA, LIBERADA_NO_SHOW, CONVERTIDA
ContratoRenta	Una unidad por contrato; extensiones por rol y anticipación; conductor adicional con licencia vigente	ABIERTO, DEVUELTO, CERRADO
InspeccionVehiculo	≥ 1 foto con sello de tiempo; firmado = inmutable; compara devolución vs entrega	—
Liquidacion	Cargos automáticos; daño sin monto hasta que Administración lo fija; disputa definitiva; depósito	Cargo: PENDIENTE_APROBACION, APLICADO, DISPUTADO, ANULADO
Vehiculo	Solo un vehículo Disponible se asigna; el km no retrocede	DISPONIBLE, ASIGNADO, RENTADO, EN_LIMPIEZA
TareaLimpieza	Una tarea abierta por devolución; solo Limpieza/Patio/Administración la completan	ABIERTA, COMPLETADA

Patrón	Dónde	Beneficio
Specification (11 reglas + AND/OR/NOT)	domain/especificaciones.py	Reglas reutilizables que explican cuál falló
State Machine (5 tablas)	domain/maquina_estados.py	Estados válidos declarados una sola vez
Domain Events (19)	domain/eventos.py + bus	Aggregates desacoplados, una transacción por aggregate
Repository + Unit of Work	infrastructure/	Dominio ignora la base; commit/rollback atómico
Ports & Adapters	application/puertos.py	Pasarela, evidencia, reloj y calendario sustituibles
CQRS básico (reportes)	infrastructure/consultas.py	SQL directo sin hidratar aggregates (ADR-002)

Flujo de eventos

ContratoAbierto → reservación CONVERTIDA, vehículo RENTADO y apertura de la liquidación con el depósito. *VehiculoDevuelto* → cálculo de cargos, vehículo EN_LIMPIEZA y tarea de limpieza. *TareaLimpiezaCompletada* → vehículo DISPONIBLE. *DepositoLiberado* → contrato CERRADO. Los eventos se publican **después del commit** y cada handler abre su propia transacción y es **idempotente**.

Decisiones de arquitectura (ADR)

ADR	Decisión
001	Domain Model + Service Layer
002	Reportes con consultas directas
003	Specification y máquina de estados declarativa
004	Un documento JSON por aggregate con columnas indexadas (SQLite y PostgreSQL)
005	Eventos en proceso, post-commit e idempotentes (Outbox como mejora)
006	API REST y contrato de errores
007	Autenticación simulada, autorización real por rol
008	Dinero con Decimal y tiempo con zona horaria
009	Configuración 12-Factor, Docker y CI
010	Adaptación del paquete de datos del PIA

5. Implementación

API REST. 32 rutas bajo */api/v1* más */salud*; OpenAPI en */docs* y */openapi.json*. Códigos: 200/201 éxito; 400 dato mal formado; 401 identidad ausente; 403 rol no autorizado o recurso ajeno; 404 no existe; 409 transición inválida o sin disponibilidad; 422 regla de negocio o esquema de entrada; 500 sin detalles internos. Cuerpo de error uniforme: *{"error": ..., "detalle": ...}*.

Seguridad de negocio. El actor se simula con *X-Actor-Id* y *X-Rol*; la autorización sí es real: cada caso de uso verifica el rol, los aggregates la vuelven a exigir en las reglas críticas y un cliente solo opera sobre sus recursos. Nunca se guarda el número de tarjeta (solo token y últimos 4 dígitos); los nombres de archivo de las fotos se sanean y nunca se sobrescribe evidencia.

Persistencia. SQLAlchemy 2 sobre PostgreSQL 16 (producción) o SQLite (desarrollo). Dinero en centavos enteros y fechas en UTC. **Configuración** por variables de entorno (*.env.example*), incluidos los parámetros de negocio.

Contenedores y CI/CD. *Dockerfile* multi-etapa (pruebas y producción) y *docker-compose.yml* con tres servicios: *db* (PostgreSQL con healthcheck), *app* y *tests* (base separada). GitHub Actions ejecuta *ruff*, la suite en Python 3.11 y 3.12 contra PostgreSQL con *--cov-fail-under=80* y *docker compose*. *render.yaml* describe el despliegue.

6. Verificación y evidencias

Nivel	Archivo	Pruebas	Qué verifica
Dominio (sin base de datos)	test_valores · test_maquina_y_especificaciones · test_politicas · test_agregados	123	Value objects, Specifications, políticas y aggregates, con valores límite (29 min, 24 h exactas, 400 km, 1/8, 48 h, 5 días hábiles)
Infraestructura	test_infraestructura	27	Serialización, repositorios, rollback, calendario, pasarela, bus, datos demo, configuración
Integración de servicios	test_servicios	55	Flujos completos con eventos, permisos por rol, idempotencia, reportes
API	test_api	15	Contrato de códigos HTTP, autenticación, flujo completo por HTTP, reloj simulado
Paquete de datos	test_paquete_datos	134	120 casos y 20 transiciones contra un oráculo independiente
Total		354	(356 con las 2 pruebas específicas de PostgreSQL)

Capa	Líneas	Cobertura de líneas	Cobertura de ramas
domain	1,046	99.6 %	97.1 %

Capa	Líneas	Cobertura de líneas	Cobertura de ramas
application	809	98.5 %	89.4 %
infrastructure	454	99.3 %	96.1 %
api	253	98.4 %	64.3 %
validacion_paquete	135	99.3 %	94.4 %
Total	2,764	98.5 %	—

Las pruebas encontraron y permitieron corregir defectos reales durante el desarrollo: (1) las fotos de daños con el mismo nombre que otra foto del checklist se sobrescribían, con riesgo para la evidencia; (2) una categoría deshabilitada se evaluaba después de la disponibilidad y devolvía un error engañoso; (3) los handlers de eventos fallaban si un evento se repetía cuando el vehículo ya había avanzado de estado; (4) los dos encabezados de seguridad compartían el mismo nombre de esquema y Swagger solo mostraba uno.

Capturas de pantalla a insertar

#	Captura	Cómo obtenerla
1	Resultado de *pytest --cov=src* con «354 passed» y cobertura 98.5 %	Terminal, desde la raíz del repositorio
2	*docker compose --profile tests up* terminando con código 0	Terminal
3	Swagger UI (*/docs*) con la lista de rutas y el botón Authorize	Navegador
4	Respuesta 403 de «aprobar extensión» como AGENTE y 200 como ADMINISTRACION	Swagger UI
5	Salida de *python scripts/demo_flujo.py* con «Demo completa»	Terminal
6	Pestaña *Actions* de GitHub con el workflow CI NovaRent en verde	GitHub
7	La aplicación desplegada en */docs* y */salud* con la URL visible	Navegador

7. Trazabilidad

La matriz completa (necesidad → historia → regla → componente → prueba → endpoint) está en *docs/trazabilidad.md*. Resumen por necesidad:

Necesidad	Historias	Componentes principales	Pruebas representativas
N1	US01, US08	ServicioDisponibilidad, contar_traslapadas	test_disponibilidad_descuenta_reservaciones_activas
N2	US06, US07, US13	InspeccionVehiculo, EvidenciaFoto	test_las_fotos_se_guardan_con_sello_de_tiempo_del_servidor
N3	US14, US15, US16	PoliticaCargos	TestCargoPorKilometraje / Combustible / Retraso
N4	US08, US09, US10	PoliticaUpgrade, Reservacion	test_sin_unidad_se_ofrece_upgrade_sin_costo…
N5	US03, US10, US11, US12	permisos.py, exigir_rol	TestPermisosPorRol
N6	US20	Liquidacion.liberar_deposito	test_deposito_se_libera_a_5_dias_habiles_sin_incidencias
N7	US18	TareaLimpieza, FlotaAppService	test_repetir_limpieza_y_cierre
N8	US06, US07, US13, US17	Liquidacion (disputas)	test_disputa_y_resolucion
N9	US19	ConsultasSQL	test_reporte_de_ocupacion_ingresos_y_rotacion
N10	US01–US20	Máquinas de estado, eventos	test_ciclo_completo_con_cargos_disputa_limpieza_y_deposito

Paquete de datos del PIA

El paquete modela «renta de equipos» y su propio LEEME indica no reemplazar los requerimientos del equipo. Se documentaron las diferencias (*docs/adaptacion_paquete_datos.md*): los CSV alimentan los datos de demostración (40 clientes, 20 categorías, 160 vehículos, 2 sucursales) y los 120 casos (53 CUMPLE y 67 NO_CUMPLE) y 20 transiciones (12 válidas y 8 rechazadas) se ejecutaron con los componentes genéricos del dominio; la evidencia por caso está en *docs/evidencias/*.

8. Registro de uso de herramientas de IA

Completar según la política del curso:

Campo	Detalle
Herramienta y versión	Claude (Anthropic)
Para qué se usó	*(describe: generación de código base, pruebas, documentación, revisión...)*
Prompt(s) principales	*(pega los prompts que usaste)*
Qué se revisó y corrigió manualmente	*(por ejemplo: validación de reglas contra el BRD, parámetros supuestos, ejecución propia de las pruebas)*
Defectos detectados por las pruebas	Ver sección 6 (4 defectos corregidos)

9. Conclusiones

Lo logrado. El diseño de la Unidad 3 se materializó sin cambiar sus decisiones de fondo: las reglas del BRD viven en el dominio como Specifications, políticas y máquinas de estado; los aggregates se coordinan con eventos idempotentes; y la API, la base de datos y la pasarela de pago quedaron como detalles intercambiables detrás de puertos. Los parámetros del negocio (24 h, 29 min, 48 h, 5 días hábiles) son configuración, no código.

Lo aprendido. (1) Un 98 % de cobertura no sustituye el análisis de bordes: las pruebas más valiosas fueron las de valores límite y las de idempotencia. (2) Modelar el tiempo y el dinero como puertos y value objects eliminó la fragilidad de las pruebas. (3) Ejecutar la misma suite contra SQLite y PostgreSQL detectó diferencias de tipos (fechas con zona y dinero) antes de llegar a producción. (4) Documentar los supuestos que el BRD no fijaba evita decidirlos en silencio dentro del código.

Limitaciones y trabajo futuro. La autenticación se simula con encabezados (JWT en producción); los eventos se publican en proceso sin patrón Outbox; no hay migraciones versionadas (Alembic) ni claves foráneas entre tablas; el cargo por conductor joven (< 25 años) se detecta pero no se cobra porque el BRD no define su monto; y los montos supuestos deben validarse con el negocio real.

Bibliografía

Evans, E. (2003). *Domain-Driven Design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Percival, H., & Gregory, B. (2020). *Architecture patterns with Python*. O'Reilly Media.

Ramírez, S. (s. f.). *FastAPI* [Documentación]. https://fastapi.tiangolo.com/

SQLAlchemy. (s. f.). *SQLAlchemy 2.0 documentation*. https://docs.sqlalchemy.org/

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Wiggins, A. (2011). *The twelve-factor app*. https://12factor.net/

Anexo A. Diagramas (código Mermaid)

Los diagramas se entregan como código Mermaid (versionable y renderizado por GitHub). Para incluir una imagen en este reporte, pega el bloque en *https://mermaid.live* y exporta PNG.

Arquitectura · 1. Capas y dependencias

```
flowchart TD
    API["API REST - FastAPI<br/>src/api"] --> APP["Application Services y DTOs<br/>src/application"]
    APP --> DOM["Dominio: Aggregates, Value Objects,<br/>Specifications, Politicas, Eventos<br/>src/domain"]
    APP -. "usa puertos" .-> PUERTOS["Puertos: UnidadDeTrabajo, Reloj, PasarelaPago,<br/>AlmacenEvidencia, BusEventos, CalendarioHabil"]
    INFRA["Infraestructura: SQLAlchemy, Repositorios, UoW,<br/>adaptadores<br/>src/infrastructure"] -. "implementa" .-> PUERTOS
    INFRA --> DOM
    COMP["Composition Root<br/>src/composicion.py"] --> API
    COMP --> INFRA
    COMP --> APP
```

Arquitectura · 2. Mapa del dominio (un Aggregate = una transacción)

```
classDiagram
    class Reservacion {
      +folio +estado +periodo +tarifa_diaria
      +cancelar() +liberar_por_no_show()
      +proponer_upgrade() +rechazar_upgrade() +otorgar_upgrade_cortesia()
    }
    class ContratoRenta {
      +id +estado +periodo +referencia_pago
      +solicitar_extension() +aprobar_extension()
      +registrar_devolucion() +cerrar()
    }
    class InspeccionVehiculo {
      +tipo ENTREGA/DEVOLUCION +fotos +danios +firma
      +firmar() +comparar_contra()
    }
    class Liquidacion {
      +cargos +deposito
      +calcular_cargos() +aprobar_cargo_danio()
      +disputar_cargo() +resolver_disputa() +liberar_deposito()
    }
    class Vehiculo { +estado +kilometraje +asignar_a_entrega() +marcar_rentado() }
    class TareaLimpieza { +estado +completar() }
    class Cliente { +licencia +metodo_pago +puede_rentar() }
    class CategoriaVehiculo { +nivel +tarifa_diaria +deposito }
    ContratoRenta "1" --> "1" Reservacion : convierte
    ContratoRenta "1" --> "1" Vehiculo : usa
    InspeccionVehiculo "2" --> "1" ContratoRenta : entrega y devolucion
    Liquidacion "1" --> "1" ContratoRenta : liquida
    TareaLimpieza "n" --> "1" Vehiculo : prepara
    Reservacion --> Cliente
    Reservacion --> CategoriaVehiculo
```

Arquitectura · 3. Máquinas de estados

```
stateDiagram-v2
    [*] --> CREADA
    CREADA --> UPGRADE_OFRECIDO: proponer_upgrade
    UPGRADE_OFRECIDO --> UPGRADE_ACEPTADO: aceptar_upgrade
    UPGRADE_OFRECIDO --> EN_LISTA_ESPERA: rechazar_a_lista_espera
    UPGRADE_OFRECIDO --> CANCELADA: rechazar_con_reembolso
    CREADA --> UPGRADE_ACEPTADO: upgrade de cortesia
    CREADA --> CONVERTIDA: contrato abierto
    UPGRADE_ACEPTADO --> CONVERTIDA: contrato abierto
    CREADA --> CANCELADA: cancelar
    CREADA --> LIBERADA_NO_SHOW: 2 h sin presentarse
    CONVERTIDA --> [*]
    CANCELADA --> [*]
    LIBERADA_NO_SHOW --> [*]
```

Arquitectura · 3. Máquinas de estados

```
stateDiagram-v2
    direction LR
    state "Contrato" as C {
        [*] --> ABIERTO
        ABIERTO --> DEVUELTO: registrar_devolucion
        DEVUELTO --> CERRADO: deposito liberado
    }
    state "Vehiculo" as V {
        [*] --> DISPONIBLE
        DISPONIBLE --> ASIGNADO: asignar_a_entrega
        ASIGNADO --> RENTADO: contrato abierto
        RENTADO --> EN_LIMPIEZA: devolucion
        EN_LIMPIEZA --> DISPONIBLE: tarea completada
    }
```

Arquitectura · 3. Máquinas de estados

```
stateDiagram-v2
    [*] --> PENDIENTE_APROBACION: dano nuevo
    [*] --> APLICADO: km, combustible o retraso
    PENDIENTE_APROBACION --> APLICADO: Administracion fija el monto
    APLICADO --> DISPUTADO: el cliente disputa
    DISPUTADO --> APLICADO: resolucion CONFIRMADO
    DISPUTADO --> ANULADO: resolucion ANULADO
```

Arquitectura · 4. Flujo de eventos (consistencia eventual entre aggregates)

```
flowchart LR
    CA["ContratoAbierto"] --> H1["Reservacion CONVERTIDA"]
    CA --> H2["Vehiculo RENTADO"]
    CA --> H3["Abre Liquidacion con deposito"]
    VD["VehiculoDevuelto"] --> H4["Calcula cargos"]
    VD --> H5["Vehiculo EN_LIMPIEZA"]
    VD --> H6["Crea TareaLimpieza"]
    TL["TareaLimpiezaCompletada"] --> H7["Vehiculo DISPONIBLE"]
    DL["DepositoLiberado"] --> H8["Contrato CERRADO"]
```

Arquitectura · 5. Secuencia: entrega y apertura del contrato

```
sequenceDiagram
    actor Ag as Agente
    participant API as API REST
    participant EA as EntregaAppService
    participant IA as InspeccionAppService
    participant CA as ContratoAppService
    participant D as Dominio
    participant P as PasarelaPago
    participant B as BusEventos
    Ag->>API: POST /reservaciones/{folio}/entrega
    API->>EA: iniciar_entrega
    EA->>D: Cliente.puede_rentar (Specifications)
    EA->>D: Vehiculo.asignar_a_entrega
    Ag->>API: POST /inspecciones/entrega (fotos)
    API->>IA: registrar_inspeccion_entrega
    IA->>D: InspeccionVehiculo.registrar
    Ag->>API: POST /inspecciones/{id}/firma (cliente)
    Ag->>API: POST /contratos
    API->>CA: abrir_contrato
    CA->>P: retener_deposito
    CA->>D: ContratoRenta.abrir
    CA->>B: publicar ContratoAbierto (despues del commit)
    B-->>D: Reservacion CONVERTIDA, Vehiculo RENTADO, abre Liquidacion
```

Arquitectura · 6. Secuencia: devolución y cargos

```
sequenceDiagram
    actor Pa as Patio
    actor Ag as Agente
    participant CA as ContratoAppService
    participant B as BusEventos
    participant LA as LiquidacionAppService
    participant D as Dominio
    Pa->>CA: checklist de devolucion
    Ag->>CA: procesar_devolucion
    CA->>D: ContratoRenta.registrar_devolucion
    CA->>B: publicar VehiculoDevuelto
    B->>LA: calcular_cargos
    LA->>D: devolucion.comparar_contra(entrega)
    LA->>D: Liquidacion.calcular_cargos (PoliticaCargos)
    D-->>LA: cargos km, combustible, retraso y danios pendientes
    B->>CA: Vehiculo EN_LIMPIEZA y TareaLimpieza
```

Arquitectura · 7. Persistencia

```
erDiagram
    clientes { string id PK }
    categorias { string id PK int nivel }
    vehiculos { string id PK string categoria_id string sucursal_id string estado }
    reservaciones { string id PK string cliente_id string categoria_id string estado datetime inicio datetime fin }
    contratos { string id PK string reservacion_folio string vehiculo_id string estado int monto_base }
    inspecciones { string id PK string contrato_id string tipo }
    liquidaciones { string id PK string contrato_id string deposito_estado int total_cargos }
    tareas_limpieza { string id PK string vehiculo_id string estado datetime creada_en }
    categorias ||--o{ vehiculos : agrupa
    clientes ||--o{ reservaciones : hace
    reservaciones ||--o| contratos : se_convierte_en
    contratos ||--o{ inspecciones : tiene
    contratos ||--|| liquidaciones : liquida
    vehiculos ||--o{ tareas_limpieza : requiere
```

Arquitectura · 8. Despliegue

```
flowchart LR
    U["Cliente HTTP / Swagger"] --> A["Contenedor app<br/>uvicorn + FastAPI"]
    A --> DB[("PostgreSQL 16")]
    A --> EV[("Volumen de evidencias")]
    T["Contenedor tests<br/>pytest + cobertura"] --> DB2[("BD novarent_test")]
    GH["GitHub Actions"] -- "ruff, pytest en PostgreSQL, docker compose" --> A
```

BPMN · 1. Reservación (US01 · US02 · US03 · US04)

```
flowchart TD
    subgraph Cliente
        A([Inicio]) --> B["Busca disponibilidad por sucursal, fechas y categoria"]
        B --> C{"Hay unidad?"}
        C -- No --> Z1([Fin: sin disponibilidad 409])
        C -- Si --> D["Genera la reservacion"]
        D --> E{"Cancela?"}
    end
    subgraph Sistema
        D --> D1["Reservacion CREADA: garantiza categoria, no unidad"]
        E -- Si --> F{"Faltan 24 h o mas?"}
        F -- Si --> G["Cancelacion sin penalizacion"]
        F -- No --> H["Cancelacion con penalizacion"]
        E -- No --> I{"Se presenta en 2 h?"}
        I -- No --> J["Job no-show: libera la reservacion"]
        I -- Si --> K([Pasa al proceso de Entrega])
    end
```

BPMN · 2. Entrega y excepciones (US05 · US06 · US07 · US08 · US09 · US10)

```
flowchart TD
    subgraph Agente
        A([Cliente llega]) --> B["Valida licencia y metodo de pago"]
        B --> C{"Cliente apto?"}
        C -- No --> Z1([Fin: rechazado 422])
        C -- Si --> D{"Hay unidad en la categoria reservada?"}
    end
    subgraph Excepcion
        D -- No --> E{"Hay categoria superior con unidad?"}
        E -- No --> Z2([Fin: sin disponibilidad 409])
        E -- Si --> F["Ofrece upgrade sin costo"]
        F --> G{"Cliente acepta?"}
        G -- No --> H["Lista de espera o reembolso"]
        G -- Si --> D2["Asigna unidad de la categoria superior"]
        Admin["Administracion: upgrade de cortesia"] -.-> D2
    end
    subgraph Patio
        D -- Si --> I["Asigna la unidad con menos km"]
        D2 --> J
        I --> J["Captura checklist: combustible, km, danios y fotos con sello de tiempo"]
    end
    subgraph Cliente
        J --> K["Firma digital del checklist"]
        K --> L["Checklist inmutable"]
    end
    subgraph Sistema
        L --> M["Retiene el deposito de garantia"]
        M --> N["Abre el contrato; reservacion CONVERTIDA; vehiculo RENTADO"]
    end
```

BPMN · 3. Extensión de renta (US11 · US12)

```
flowchart TD
    A([Cliente pide mas tiempo]) --> B{"Faltan 2 h o mas para vencer?"}
    B -- No --> Z1([Rechazada 422])
    B -- Si --> C{"Hay disponibilidad en el nuevo lapso?"}
    C -- No --> Z2([Rechazada 409])
    C -- Si --> D{"Horas solicitadas"}
    D -- "48 h o menos" --> E["Agente aprueba: renta extendida"]
    D -- "Mas de 48 h" --> F["Queda PENDIENTE de autorizacion"]
    F --> G{"Administracion aprueba antes del vencimiento?"}
    G -- Si --> E
    G -- No --> Z3([Sin extension])
```

BPMN · 4. Devolución, cargos y disputa (US13 · US14 · US15 · US16 · US17)

```
flowchart TD
    subgraph Patio
        A([Cliente devuelve la unidad]) --> B["Captura checklist de devolucion con fotos"]
    end
    subgraph Agente
        B --> C["Procesa la devolucion: contrato DEVUELTO"]
    end
    subgraph Sistema
        C --> D["Compara devolucion contra entrega"]
        D --> E["Cargo por kilometraje: sobre 200 km por dia a 3 MXN por km"]
        D --> F["Cargo por combustible: mas de 1/8 de tanque"]
        D --> G["Cargo por retraso: despues de 29 min, por hora, tope de 1 dia"]
        D --> H["Danios nuevos: cargo PENDIENTE de monto"]
        E & F & G & H --> I["Liquidacion con cargos"]
    end
    subgraph Administracion
        H --> J["Fija el monto del danio"]
    end
    subgraph Cliente
        I --> K{"Disputa un cargo?"}
        J --> K
        K -- Si --> L["Disputa con motivo"]
    end
    subgraph Disputa
        L --> M["Se compara la evidencia de entrega y devolucion"]
        M --> N["Administracion resuelve: CONFIRMADO o ANULADO"]
    end
```

BPMN · 5. Cierre: limpieza, disponibilidad y depósito (US18 · US19 · US20)

```
flowchart TD
    subgraph Sistema
        A([Vehiculo devuelto]) --> B["Vehiculo EN_LIMPIEZA y se crea la tarea de limpieza"]
        L["Job diario"] --> M{"5 dias habiles cumplidos y sin incidencias?"}
        M -- Si --> N["Libera el deposito en la pasarela"]
        N --> O["Contrato CERRADO"]
        M -- No --> P["Retiene el deposito"]
    end
    subgraph Limpieza
        B --> C["Prepara la unidad"]
        C --> D["Marca la tarea como completada"]
    end
    subgraph Flota
        D --> E["Vehiculo DISPONIBLE y evento VehiculoListoParaRenta"]
    end
    subgraph Administracion
        R["Consulta reporte: ocupacion, ingresos y rotacion"]
    end
```